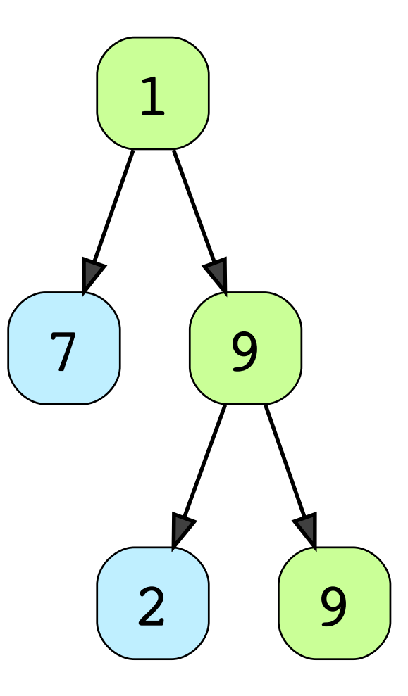
Solution: Path With Given Sequence

Problem Statement

Given a binary tree and a number sequence, find if the sequence is present as a root-to-leaf path in the given tree.

Example 1:

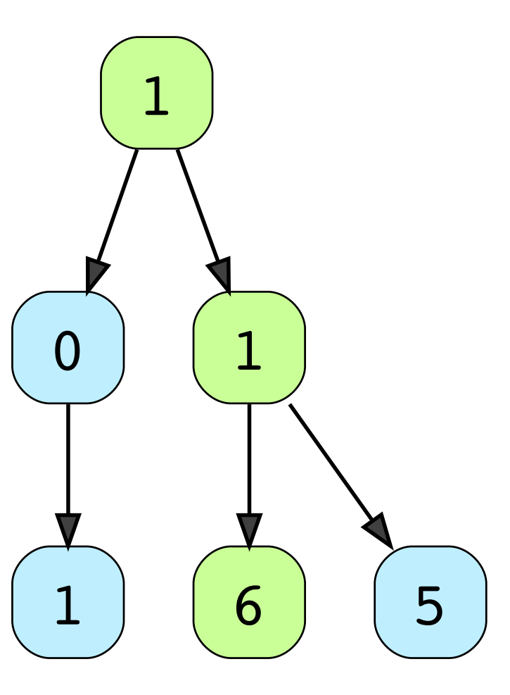
Sequence: [1, 9, 9]
Output: true
Explanation: The tree has a path 1 -> 9 -> 9.



Example 2:

Sequence: [1, 0, 7]
Output: false
Explanation: The tree does not have a path 1 -> 0 -> 7.

Sequence: [1, 1, 6]
Output: true
Explanation: The tree has a path 1 -> 1 -> 6.



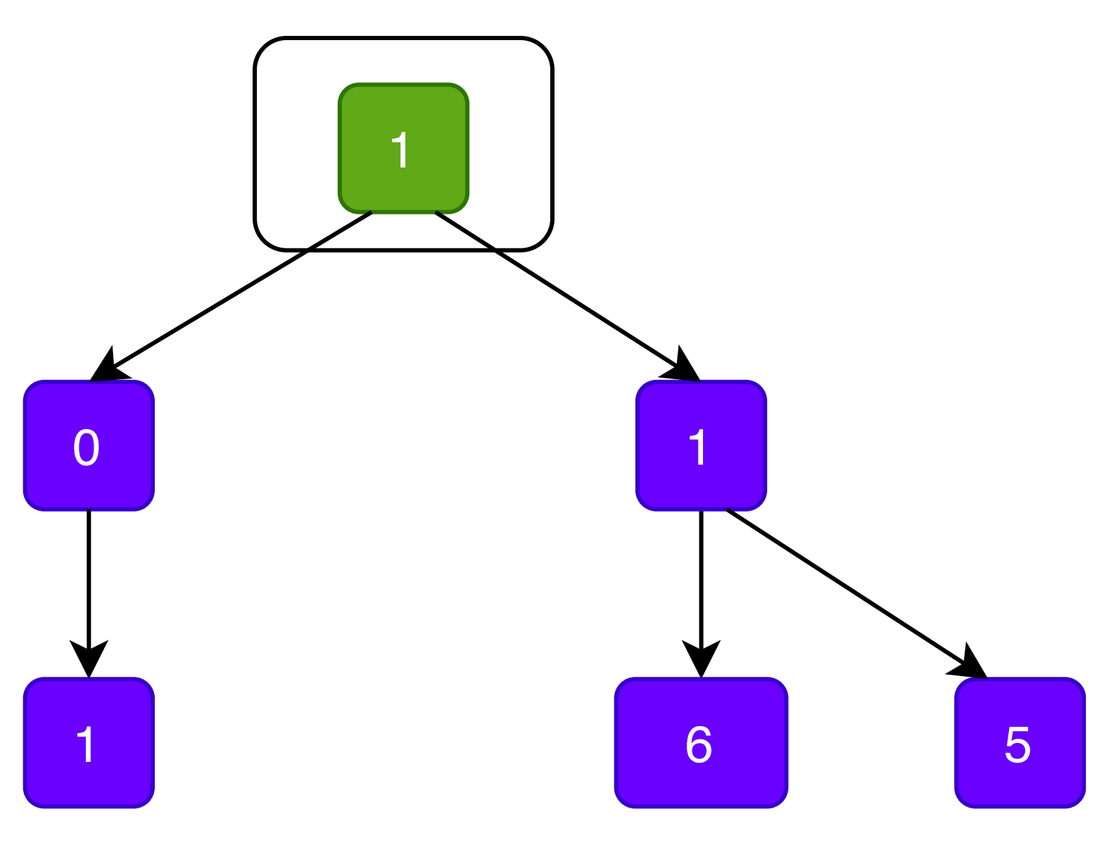
Solution

This problem follows the **Binary Tree Path Sum** pattern. We can follow the same **DFS** approach and additionally, track the element of the given sequence that we should match with the current node. Also, we can return `false` as soon as we find a mismatch between the sequence and the node value.

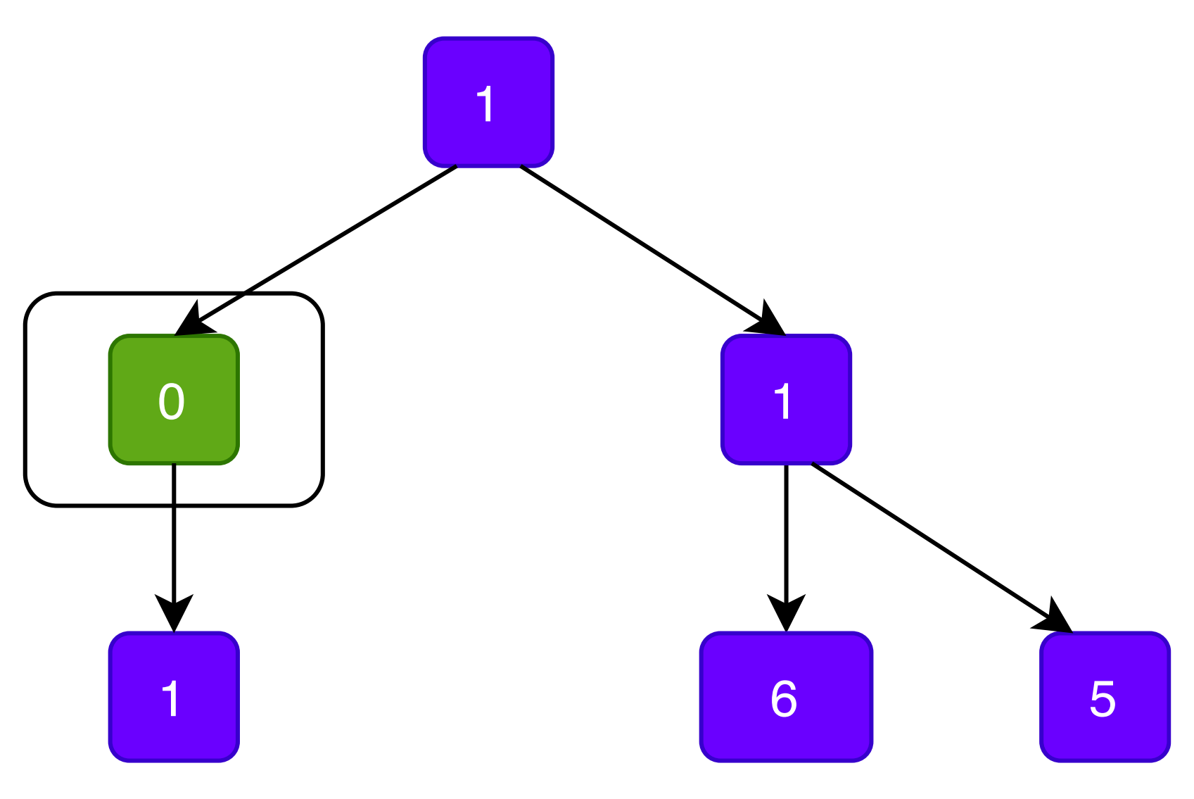
Here is the visual representation of the algorithm:

Example: 1

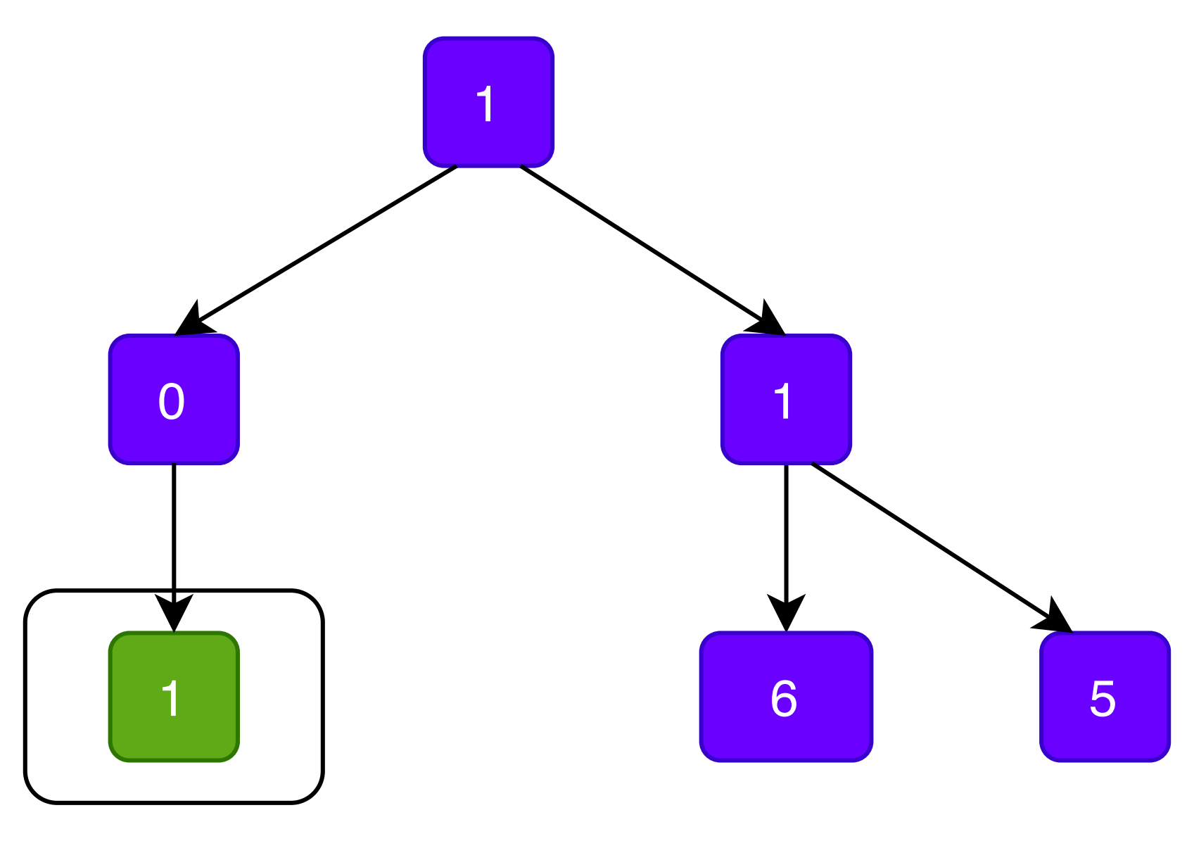
Seq: [1, 0, 7]



Seq: [1, 0, 7]



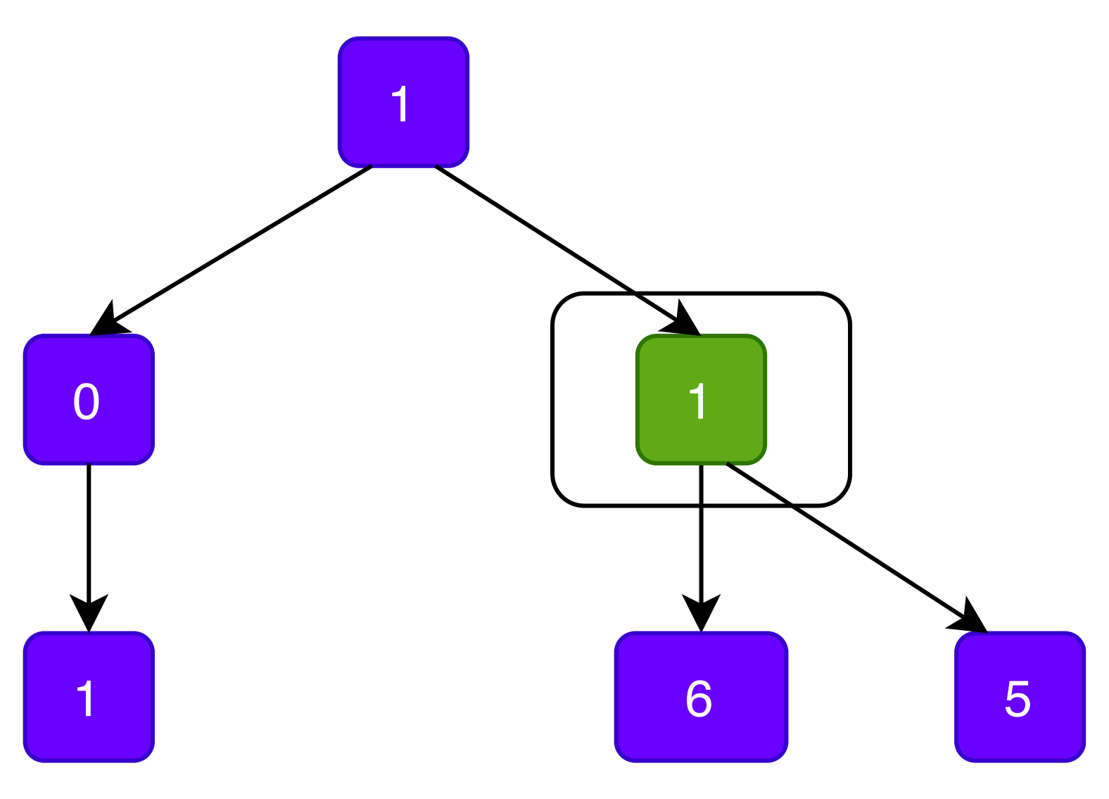
Seq: [1, 0, 7]



Now 1 is the last node and it does not match the provided squence last element so false will be returned

Now, we will backtrack and will reach to the 1 at the right

Seq: [1, 0, 7]

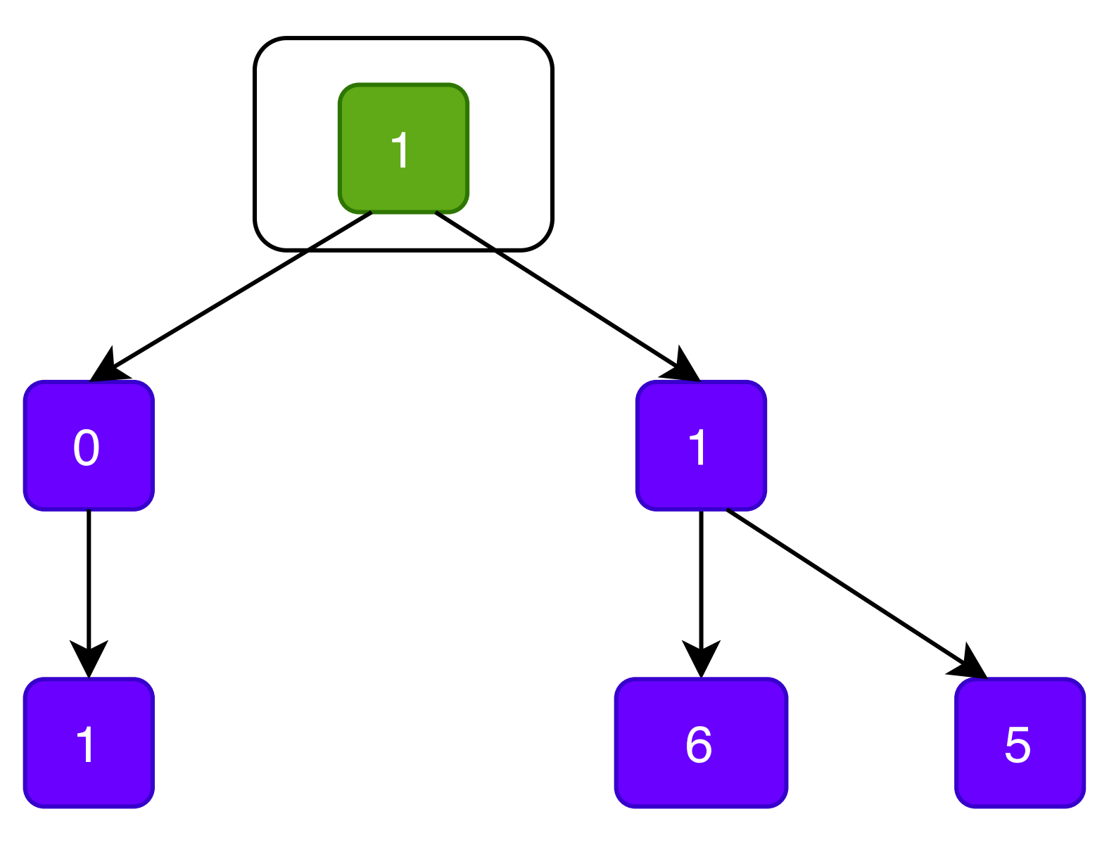


Now 1 does not match the sequence second element so false will be returned and we will not go further

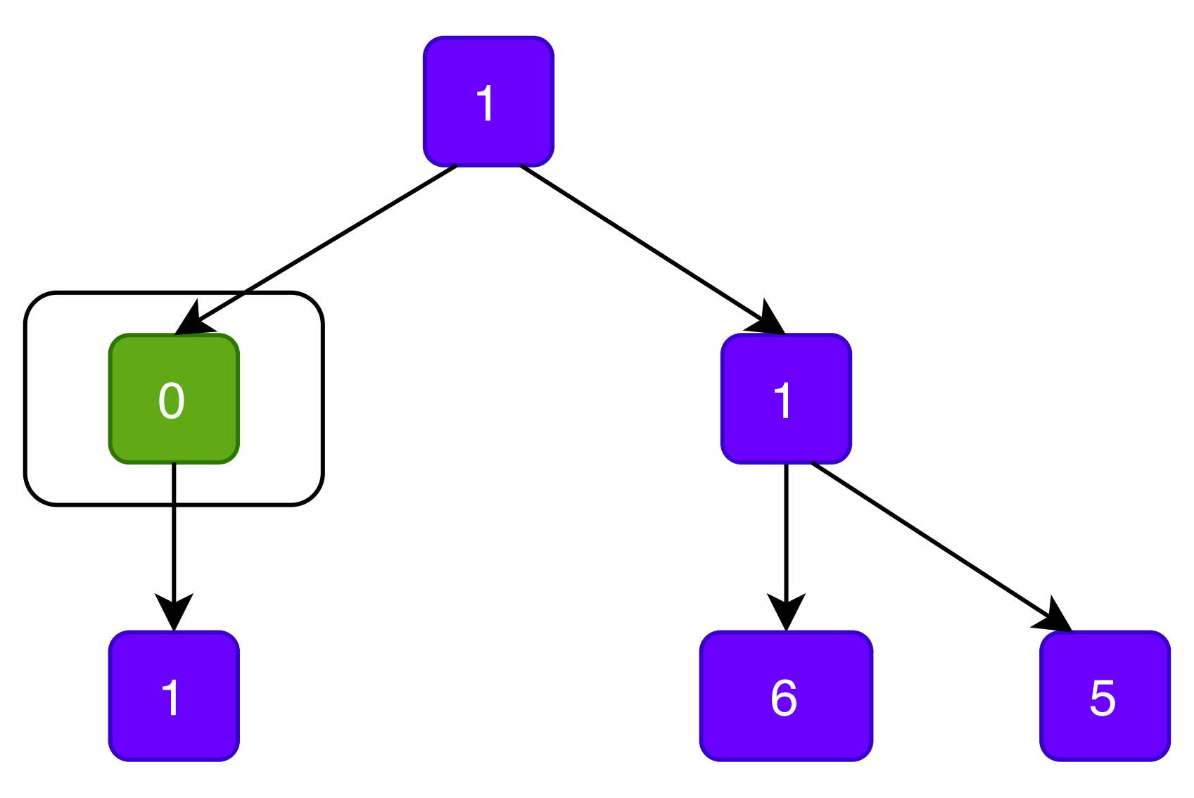
Output: false

Example: 2

Seq: [1, 1, 6]



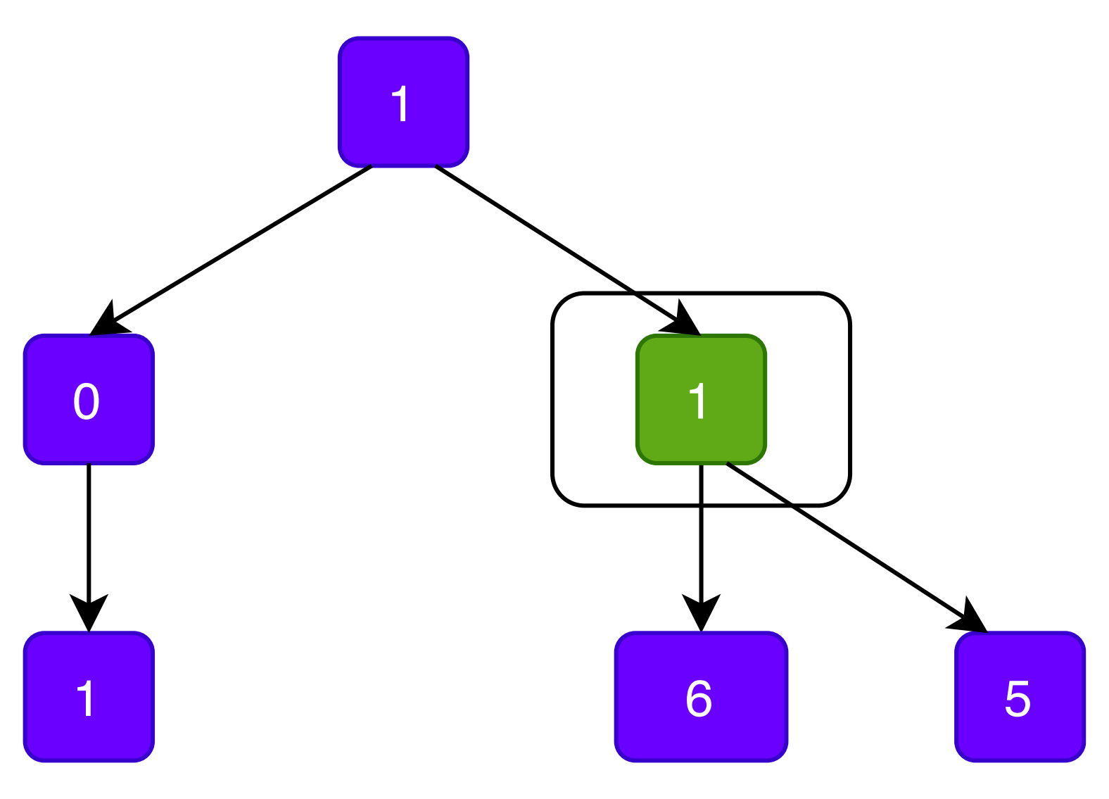
Seq: [1, 1, 6]



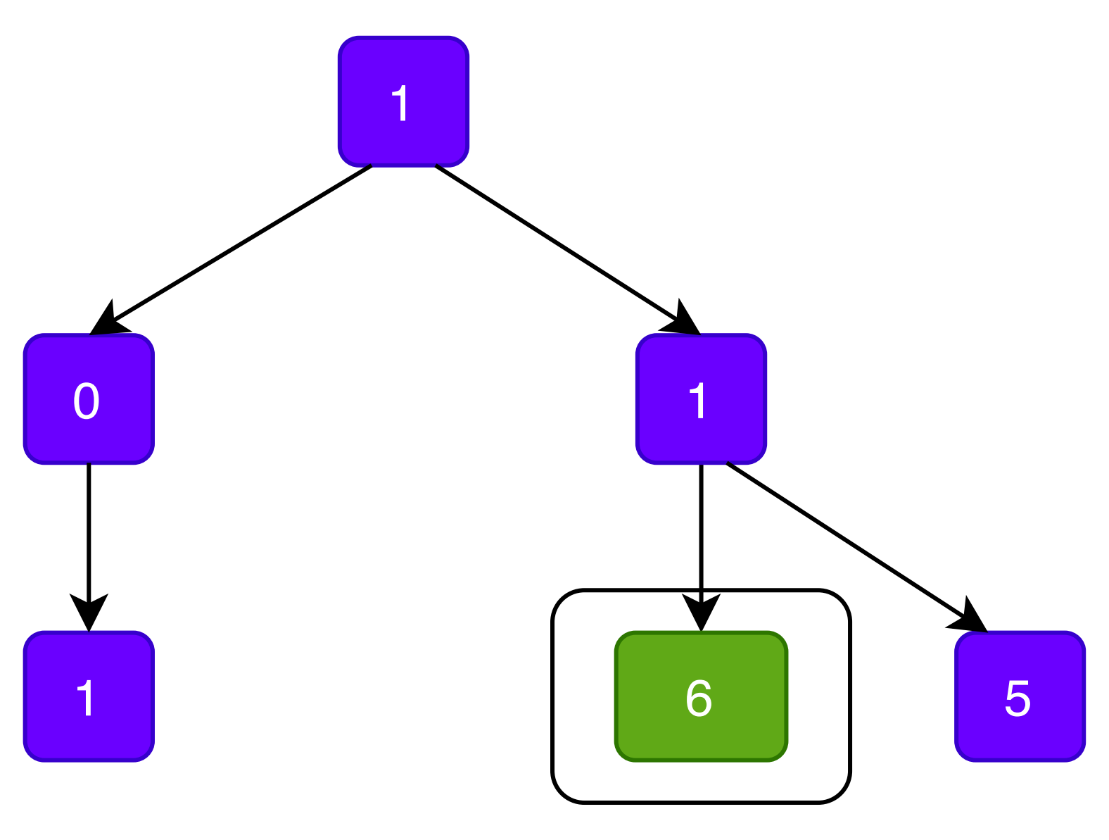
Now 0 does not match the sequence second element so false will be returned and we will not go further in that path

Now, we will backtrack and will reach to the 1 at the right

Seq: [1, 1, 6]



Seq: [1, 1, 6]



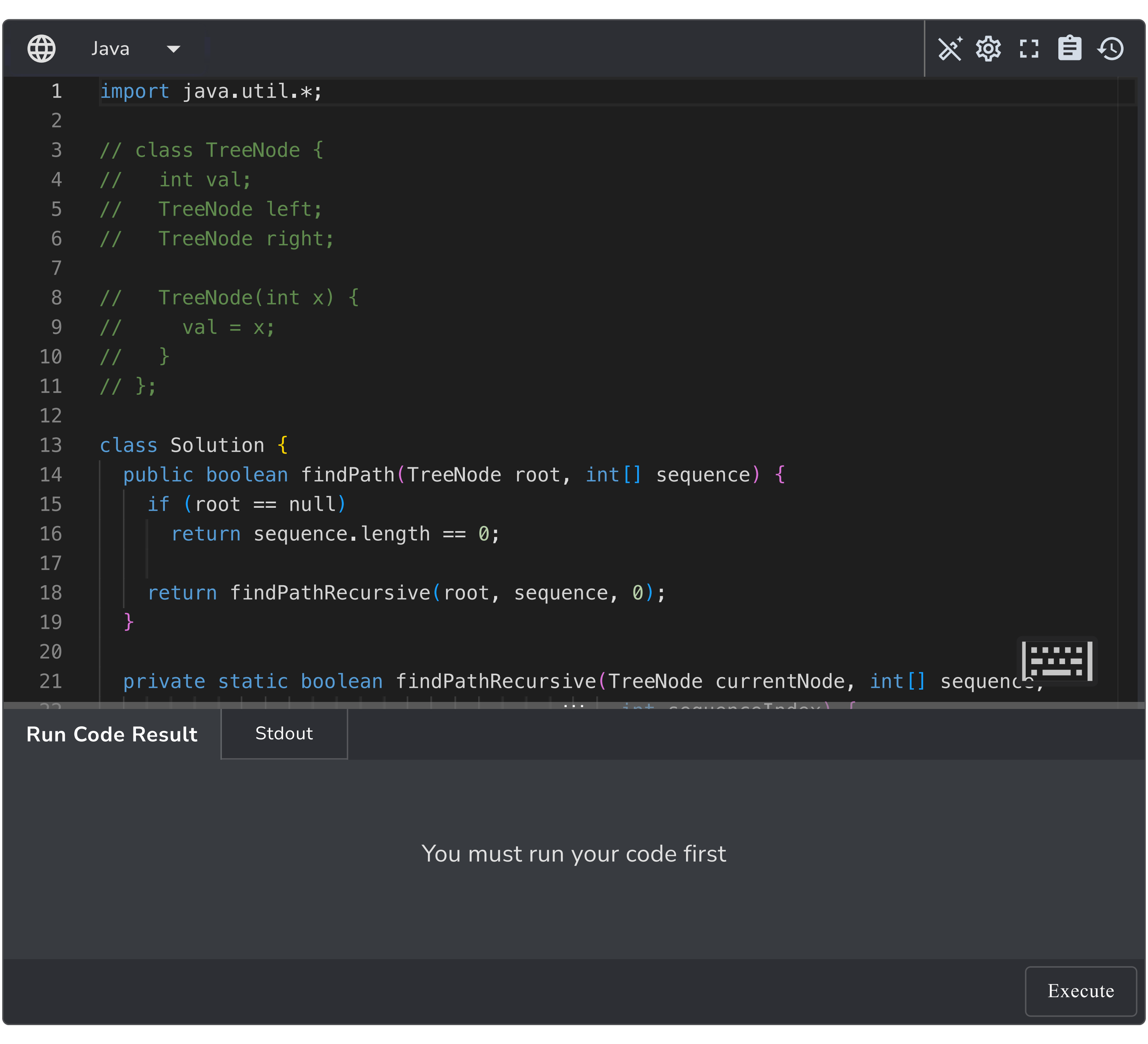
Now 6 has no further nodes and all path elements also matches the whole sequence so we will return true here

Output: true

Sum of Path Nums

Code

Here is the code for this algorithm:



```
import java.util.*;

// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;

//   TreeNode(int x) {
//     val = x;
//   }
// };

class Solution {
  public boolean findPath(TreeNode root, int[] sequence) {
    if (root == null)
      return sequence.length == 0;

    return findPathRecursive(root, sequence, 0);
  }

  private static boolean findPathRecursive(TreeNode currentNode, int[] sequence,
```

You must run your code first

Time Complexity

The time complexity of the above algorithm is O(N), where ‘N’ is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space Complexity

The space complexity of the above algorithm will be O(N) in the worst case. This space will be used to store the recursion stack. The worst case will happen when the given tree is a linked list (i.e., every node has only one child).

← Previous
Path With Given Sequence (Medium)
Next →
Count Paths For A Sum (Medium)
☐ Mark as Completed